CS 2110 Homework 2
Digital Logic and the ALU

Max Everest, Jonathan Marto, Sebastian Jankowski, Annelise Lloyd, Zaid Akkawi

Summer 2023

Contents

1 Overview

1.1 Purpose

You have learned about digital logic, including transistors, gates, and combinational logic. Gates (AND, OR, etc.) can be built using transistors, and Combinational Logic circuits (decoder, MUX, etc.) can be built using gates. Note how the concepts build up from transistors to gates to combinational logic. We have provided you with a tool called Circuitsim (available through Docker) that allows you to simulate building circuits, without actually using physical hardware.

The purpose of this assignment is for you to become proficient building gates out of transistors, and combinational logic circuits out of gates, using Circuitsim. You will put these components together to create an ALU (arithmetic logic unit) circuit, which will allow you to perform several specified math and logic operations using digital logic.

1.2 Task

You will complete four Circuitsim files, and build an ALU from the ground up, starting with transistors, and then gates, and then the remainder of your combinational logic. Please read this entire document for detailed instructions, including which digital logic components you are allowed to use or prohibited from using for each part of the assignment.

This document also contains tutorials on using Circuitsim and creating subcircuits, among other topics.

The steps to complete this assignment include:

1. Create the standard logic gates (NAND, NOR, NOT, AND, OR)
2. Apply DeMorgan's Law to convert a provided circuit to one without any OR Gates
3. Create an 8-input multiplexer and an 8-output decoder
4. Use multiplexers to create a circuit that evaluates the sign of a number
5. Create a 1-bit full adder
6. Create an 8-bit full adder using the 1-bit full adder
7. Use your 8-bit full adder and other components to construct an 8-bit ALU

The assignment is due by 11:59 PM on June 5th, 2023. You could also submit the assignment by June 6th, 2023 for a late penalty of 25% of your overall grade. Before you start the assignment, be sure to read the rest of the PDF and the 'Resources' section.

1.3 Criteria

You must utilize the provided CS 2110 version of CircuitSim, which is available via Docker or the JAR on Canvas (See more instructions in 'Running the Autograder'). Utilizing other versions of CircuitSim is strictly prohibited and may result in damaged or corrupted files.

You will submit four Circuitsim files that you produce to Gradescope: gates.sim, demorgan.sim, plexers.sim, and alu.sim. Your circuits must work properly and produce the desired results in order to receive credit. For the ALU portion, we will give partial credit for each operation that works properly.

Be sure to avoid using components that are disallowed for each phase of this assignment.

2 Optional Tutorial

Note: This tutorial is optional and to help you get acquainted with CircuitSim before you start this homework. If you're comfortable with this software, feel free to skip to section 3. CircuitSim is an interactive circuit simulation package. We will be using this program for the next couple of homework assignments. This is a tutorial to help you get acquainted with the software. CircuitSim is a powerful simulation tool designed for educational use. This gives it the advantage of being a little more forgiving than some of the more commercial simulators. However, it still requires some time and effort to be able to use the program efficiently. With this in mind, we present you with the following assignment:

2.1 Part 1 — Read Resources

Read through the following resources

- CircuitSim Wires Documentation `https://ra4king.github.io/CircuitSim/docs/wires/`
- Tutorial 1: My First Circuit `https://ra4king.github.io/CircuitSim/tutorial/tut-1-beginner`

2.2 Part 2 — Complete Tutorial 2

Complete Tutorial 2 `https://ra4king.github.io/CircuitSim/tutorial/tut-2-xor`

Instead of saving your file as **xor.sim**, save your file as **part1.sim**. As well, make sure you label your two inputs **a** and **b**, and your output as **c**, as well as rename your subcircuit to xor.

2.3 Part 3 — Complete Tutorial 3

Complete Tutorial 3 `https://ra4king.github.io/CircuitSim/tutorial/tut-3-tunnels-splitters`

Name the subcircuit **umbrella**, the input **in**, and the output **out**. Save your file as **part2.sim**.

3 Instructions

For this assignment, you will be using CircuitSim. The version is included in the 2110 Docker container. If you are having issues with Docker, it can also be found on canvas under Files -> Tools -> CircuitSim.jar.

3.1 Requirements

For the first part of this assignment (the logic gates), you may use **only those components found in the Wiring tab** (being the input/output pins, constants, probes, clocks, splitters, tunnels, and transistors), along with any of the sub-circuits that you create or that already exist.

For the second part of this assignment (using DeMorgan's law), you may use **only those components found in the Wiring and Gates tabs (except for OR/NOR/XOR/XNOR gates)**.

For the third part of this assignment (the decoder and multiplexer), you may use **only those components found in the Wiring and Gates tabs** along with any of the sub-circuits that you create or that already exist, except in the sign evaluation circuit where you may use the **Plexer Tab** as well.

For the last part of this assignment, you may use only those components found in the **Wiring and Gates tabs** as well as any of the sub-circuits that you create or that already exist. The term sub-circuit refers to any circuits that you have constructed in this part of the homework. If you're unsure on how to utilize a

sub-circuit, check out *Section 6.* **For the ALU portion specifically, you may use the Plexer Tab as well.**

Use of anything not listed above will result in heavy deductions. Your need to have everything in their correctly named sub-circuit. More information on sub-circuits is given below.

Use tunnels where necessary to make your designs more readable, but do not overdo it! For gates, muxes, adders, and decoders you can often get clean circuits just by placing your components well rather than using tunnels everywhere.

3.2 Part 1: 1-Bit Logic Gates

ALLOWED COMPONENTS: Wiring Tab and Circuits Tab

All of the circuits in this file are in the *gates.sim* file.

For this part of the assignment, you will create a transistor-level implementation of the NAND, NOT, NOR, AND, and OR logic gates.

Remember for this section that you are only allowed to use the components listed in the Wiring section, along with any of the logic gates you are implementing in CircuitSim. For example, once you implement the NOT gate you are free to use that subcircuit in implementing other logic gates. Implementing the gates in the order of the subcircuit tabs can be the easiest option.

As a brief summary of the behavior of each logic gate:

NAND (Inputs: A, B - Output: OUT)

A	B	A NAND B
0	0	1
0	1	1
1	0	1
1	1	0

NOR (Inputs: A, B - Output: OUT)

A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

AND (Inputs: A, B - Output: OUT)

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

OR (Inputs: A, B - Output: OUT)

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

NOT (Input: IN - Output: OUT)

A	NOT A
0	1
1	0

Hint: Start by creating the NAND and NOT gates from transistors. Then use this gate as a subcircuit for implementing the others.

All of the logic gates must be within their named sub-circuits.

3.3 Part 2: DeMorgan's Law

ALLOWED COMPONENTS: Wiring Tab and Gates Tab

BANNED COMPONENTS: OR Gate, NOR Gate, XOR Gate, XNOR Gate

The circuits in this file are in the *demorgan.sim* file.

De Morgan's laws help relate conjunctions (AND) and disjunctions (OR) of propositions with negation.

In this file, you will be creating a circuit that is an alternative to the circuit that is provided in the *Provided* subcircuit (first tab). Your work will be done in the *Student* subcircuit (second tab).

3.3.1 Provided

Nothing needs to be done in this subcircuit! You may find it helpful to convert this circuit into a sum of products (**wink**). Also, feel free to test some inputs and build a truth table (**double wink**).

3.3.2 Student

This circuit needs to be directly equivalent to the circuit provided in the *Provided* subcircuit, meaning the **same inputs** should lead to the **same outputs**.

Here's the catch: There should be **ZERO** OR gates in your circuit!

We will see later in this course that OR operations may not be given for us to use and so converting boolean expressions to equivalent ones that instead use AND operations is a useful technique.

3.4 Part 3: Plexers

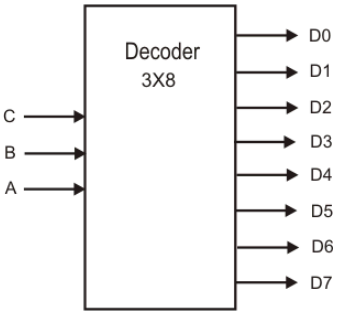
ALLOWED COMPONENTS: Wiring Tab, Circuits Tab, and Gates Tab

NOTE: For the sign evaluation circuit, you may use the Plexer tab as well.

All of the circuits in this file are in the *plexers.sim* file.

3.4.1 Decoder

The decoder you will be creating has a single 3-bit selection input (`SEL`), and eight 1-bit outputs (labeled `A`, `B`, `C`, ..., `H`). The decoder uses the `SEL` input to raise a specific output line, as seen below.

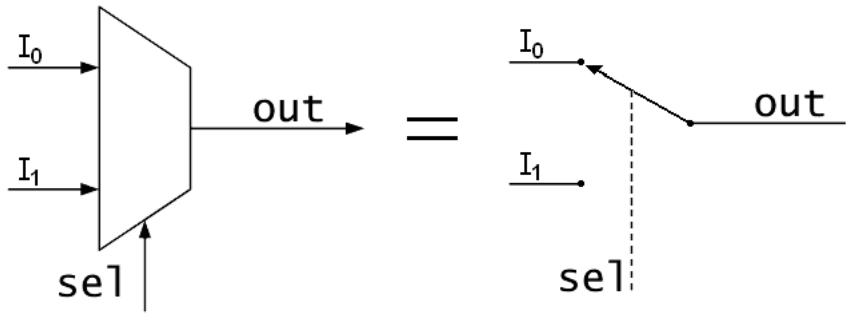


For example:

```
SEL = 000 ==> A = 1, BCDEFGH = 0
SEL = 001 ==> B = 1, ACDEFGH = 0
SEL = 010 ==> C = 1, ABDEFGH = 0
SEL = 011 ==> D = 1, ABCEFGH = 0
SEL = 100 ==> E = 1, ABCDFGH = 0
SEL = 101 ==> F = 1, ABCDEGH = 0
SEL = 110 ==> G = 1, ABCDEFH = 0
SEL = 111 ==> H = 1, ABCDEFG = 0
```

3.4.2 Multiplexer

The multiplexer you will be creating has 8 1-bit inputs (labeled appropriately as `A`, `B`, `C`, ..., `H`), a single 3-bit selection input (`SEL`), and one 1-bit output (`OUT`). The multiplexer uses the `SEL` input to choose a specific input line for forwarding to the output.



For example:

```
SEL = 000 ==> OUT = A
SEL = 001 ==> OUT = B
```

```
SEL = 010 ==> OUT = C
SEL = 011 ==> OUT = D
SEL = 100 ==> OUT = E
SEL = 101 ==> OUT = F
SEL = 110 ==> OUT = G
SEL = 111 ==> OUT = H
```

3.4.3 Sign Evaluation

ALLOWED COMPONENTS: Wiring Tab, Circuits Tab, Gates Tab, and Plexer Tab

In this step, you will construct a circuit that takes an 8-bit two's complement input and evaluates whether the input is negative, zero, or positive. Based on this, this circuit will output a 3-bit bit-vector called NZP where the most significant bit is 1 *iff* the input is negative, the middle bit is 1 *iff* the input is zero, and the least significant bit is 1 *iff* the input is positive. Only 1 bit of the output should be set at any given time. Zero is not considered a positive number. **For example, if the input to this circuit is positive, then it will output 001**

With that in mind, set the correct bit and **implement this circuit in the Sign Evaluation subcircuit**

Hint: Recall that in two's complement, the most significant bit can be used to determine the sign of a number!

3.5 Part 4: Adders & ALUs

ALLOWED COMPONENTS: Wiring Tab, Circuits Tab, and Gates Tab

NOTE: For the ALU specifically, you may use the Plexer tab as well.

All of the circuits in this file are in the *alu.sim* file.

3.5.1 1-Bit Adder

The full adder has three 1-bit inputs (`A`, `B`, and `CIN`), and two 1-bit outputs (`SUM` and `COUT`). The full adder adds `A + B + CIN` and places the sum in `SUM` and the carry-out in `COUT`.

For example:

```
A = 0, B = 1, CIN = 0 ==> SUM = 1, COUT = 0
A = 1, B = 0, CIN = 1 ==> SUM = 0, COUT = 1
A = 1, B = 1, CIN = 1 ==> SUM = 1, COUT = 1
```

Hint: Making a truth table of the inputs will help you.

3.5.2 8-Bit Adder

For this part of the assignment, you will daisy-chain 8 of your 1-bit full adders together in order to make an 8-bit full adder.

This circuit should have two 8-bit inputs (`A` and `B`) for the numbers you're adding, and one 1-bit input for `CIN`. The reason for the `CIN` has to do with using the adder for purposes other than adding the two inputs.

There should be one 8-bit output for `SUM` and one 1-bit output for `COUT`.

3.5.3 Basic 8-Bit ALU

ALLOWED COMPONENTS: Wiring Tab, Circuits Tab, Gates Tab, and Plexer Tab

You will first create a simple 8-bit ALU, using the 8-bit full adder you created previously.

For this ALU, we will be using a multiplexer. This ALU has two **8-bit** inputs for `A` and `B` and one **2-bit** input for `OP`, the op-code for the operation in the list below. It has one **8-bit** output named `OUT`. The following 4 operations will be selected from:

00. Addition	`[A + B]`
01. AND	`[A & B]`
10. NOT	`[NOT A]`
11. Pass	`[PASS A]`

NOTE: the **Pass** operation simply refers to outputting the value of A unchanged.

Notice that **NOT** and **Pass** only operate on the `A` input. **They should NOT rely on `B` being a particular value.**

The provided autograder will check the op-codes according to the order listed above (Addition (`00`), AND (`01`), etc.) and thus it is important that the operations are in this exact order.

3.5.4 Intermediate 8-Bit ALU

ALLOWED COMPONENTS: Wiring Tab, Circuits Tab, Gates Tab, and Plexer Tab

You will next create a different 8-bit ALU, with identical structure as the previous, but more complex operations as outlined below:

00.	isDouble	`[A == 2 * B]`
01.	isDivisibleBy16	`[A % 16 == 0]`
10.	multiplyBy9	`[A * 9]`
11.	maximum	`[max(A, B)]`

IMPORTANT NOTE:

Notice that **multiplyBy9** and **isDivisibleBy16** only operate on the `A` input. **They should NOT rely on `B` being a particular value.**

For the **isDouble** operation, assume that `A` and `B` will both be unsigned values.

For the **multiplyBy9** and **isDouble** operations, although there is potential for overflow, we don't need to worry about it in this homework.

For the **isOctuple** and **isDivisibleBy16** operations, return `00000001` if the condition is true, and `00000000` otherwise.

The provided autograder will check the op-codes according to the order listed above (isOctuple (`00`), isDivisibleBy16 (`01`), multiplyBy9 (`10`), maximum (`11`)) and thus it is important that the operations are in this exact order.

3.6 Running the Autograder

Note: For this homework, as you will be using circuit-sim in the docker-run website, we recommend you to run the provided autograder in the docker website's terminal (which I'll call docker terminal).

The following instructions (for the docker terminal) assume that your **'cs2110docker.sh'** shell script is in the same directory as your **'homework files'**. If it isn't, move the '.sh' file into the folder where you have your homeworks, then run it again (See docker instructions pdf/website on canvas).

To run the autograder on docker terminal, follow these instructions...

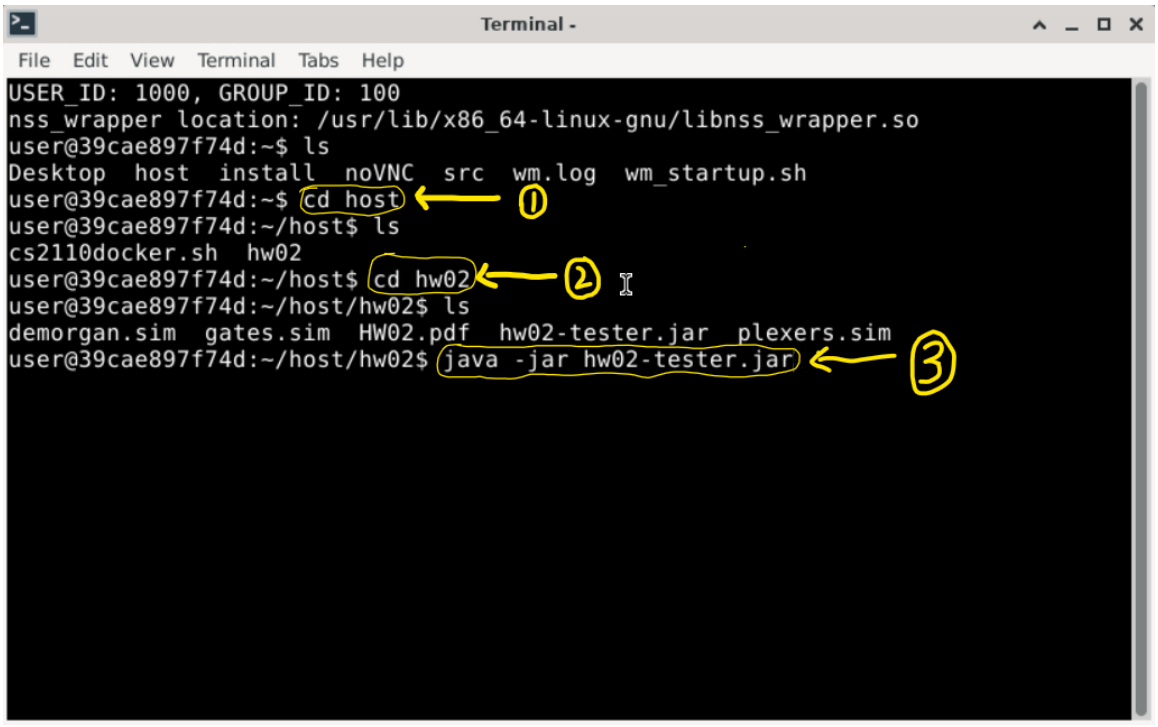
1. Open the website
2. Click 'Terminal'

(a) 

3. cd into your 'hw02' folder
4. Then, type the following command

```
java -jar hw02-tester.jar
```

5. The following visual can be followed as well.

(a) 

Make sure all the tests have been passed. Keep in mind that even if you get full credit from the autograder, we reserve the right to test for more cases. Note: To guarantee that the autograder runs without error, run it from your Docker container. To do this, either run `./cs2110docker.sh` and open the terminal inside the graphical client, or run `./cs2110docker.sh -it` to open a shell directly in your terminal.

4 Disclaimers

The following are trivial things that may lead to errors on the autograder, even though your circuit is *technically* correct.

1. Renaming or removing input pins may cause issues with the autograder because it's looking for input pins with a specific label.
2. You must use constant pins when necessary. Using any input pins for a scenario where a fixed constant value is required will lead to the autograder setting the input pin to 0 and thus cause issues for scenarios where it needed to be a constant 1.

5 Deliverables

Please upload the following files onto the assignment on Gradescope:

1. `gates.sim` — `NOT, NAND, NOR, AND, OR`
2. `demorgan.sim` — `Provided, Student`
3. `plexers.sim` — `Decoder, MUX, Sign Evaluation`
4. `alu.sim` — `1-Bit Adder, 8-Bit Adder, Basic 8-Bit ALU, Intermediate 8-Bit ALU`

Be sure to check your score to see if you submitted the right files, as well as your email frequently until the due date of the assignment for any potential updates.

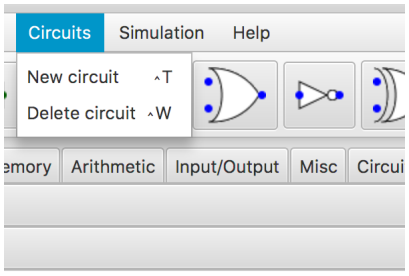
No partial credit will be given for incorrect outputs for Part 1, Part 2, Part 3, and the adder-portion of Part 4. For the ALUs, partial credit will be awarded on a per-operation basis, wherein each operation must perform successfully to be awarded credit. Because of this, we urge you to check your score before the due date.

6 Sub-Circuit Tutorial

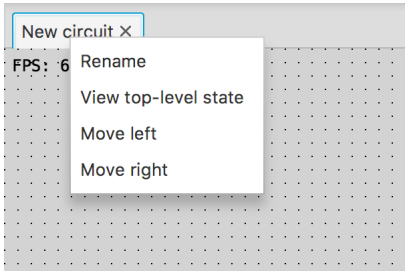
As you build circuits that are more and more sophisticated, you will want to build smaller circuits that you can use multiple times within larger circuits. Sub-circuits behave like classes in Object-Oriented languages. Any changes made in the design of a sub-circuit are automatically reflected wherever it is used. The direction of the IO pins in the sub-circuit correspond to their locations on the representation of the sub-circuit.

To create a sub-circuit:

1. Go to the "Circuits" menu and choose "New circuit"

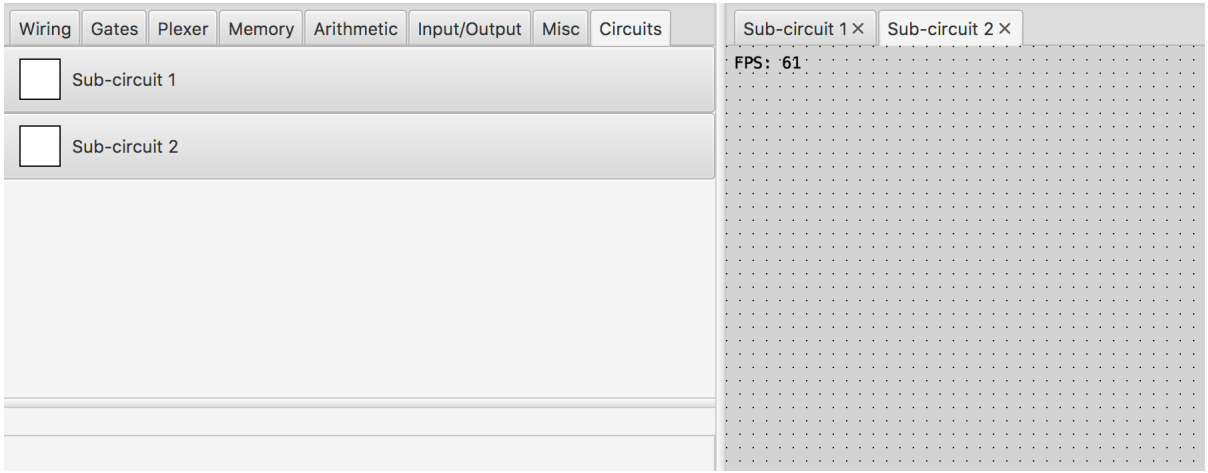


2. Name your circuit by right-clicking on the "New circuit" item and selecting "Rename"

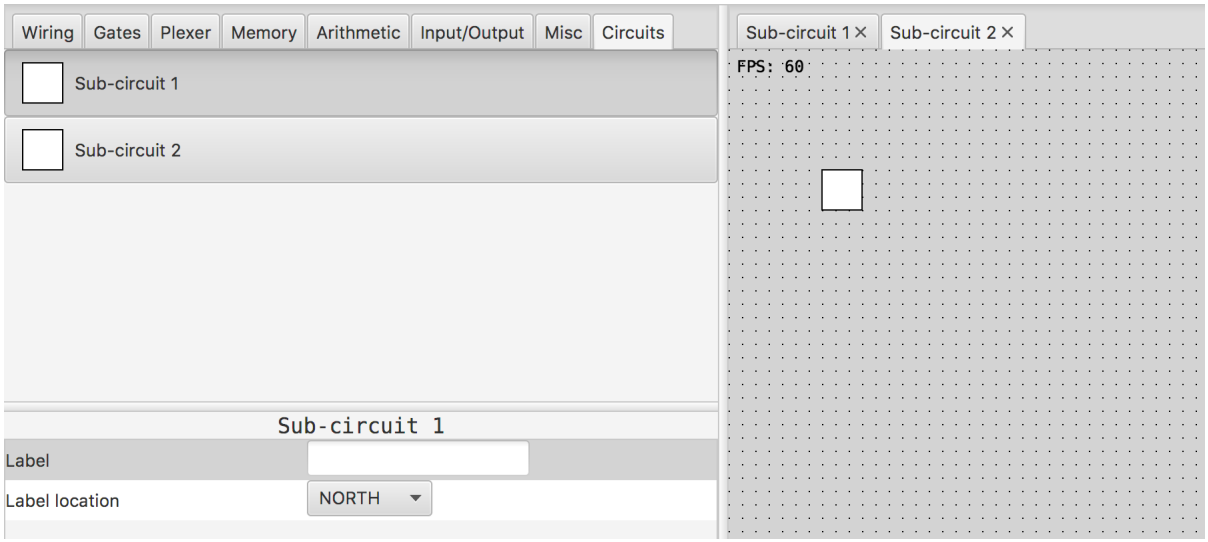


To use a sub-circuit:

1. Click the "Circuits" tab next to the "Misc" tab



2. Select the circuit you wish to use and place it in your design



7 Resources

The following links to the lecture and lab slides will help you complete the homework.

Note: this is not the complete list and you should definitely check out later slides/guides/lecture notes as well (for more info on content)

Lecture Slides:

1. **L02, all slides: Understand twos-complement, binary to decimal conversion, binary addition**

2. **L03, all slides: Understand bitwise operations, bit shifting**

3. **L04, p.28-p.73: Understand transistors, logic gates, DeMorgan's law, decoders, multiplexors, adders**

Lab Slides

1. **L02, all slides: Understand twos-complement, bitwise operations, bit shifting**

   L03, all slides: Understand P-type, N-type transistors, logic gates

   L04, all slides: Understand P-type, N-type transistors, Multiplexers, Decoders

Lab Guides:

1. **Please access all lab guides here → Lab Guides Link**

Ed discussion

1. **Checkout the HW02 - Tips and Tricks pinned post on ed discussion for more help (It will be released on Jan 27th, 2023)**

8 Rules and Regulations

8.1 General Rules

1. **Starting with the assembly homeworks, any code you write must be meaningfully commented. You should comment your code in terms of the algorithm you are implementing; we all know what each line of code does.**
2. **Although you may ask TAs for clarification, you are ultimately responsible for what you submit. This means that (in the case of demos) you should come prepared to explain to the TA how any piece of code you submitted works, even if you copied it from the book or read about it on the internet.**
3. **Please read the assignment in its entirety before asking questions.**
4. **Please start assignments early, and ask for help early. Do not email us the night the assignment is due with questions.**
5. **If you find any problems with the assignment it would be greatly appreciated if you reported them to the author (which can be found at the top of the assignment). Announcements will be posted if the assignment changes.**

8.2 Submission Conventions

1. **All files you submit for assignments in this course should have your name at the top of the file as a comment for any source code file, and somewhere in the file, near the top, for other files unless otherwise noted.**
2. **When preparing your submission you may either submit the files individually to Canvas/Gradescope or you may submit an archive (zip or tar.gz only please) of the files. You can create an archive by right clicking on files and selecting the appropriate compress option on your system. Both ways (uploading raw files or an archive) are exactly equivalent, so choose whichever is most convenient for you.**
3. **Do not submit compiled files, that is .class files for Java code and .o files for C code. Only submit the files we ask for in the assignment.**
4. **Do not submit links to files. The autograder does not understand it, and we will not manually grade assignments submitted this way as it is easy to change the files after the submission period ends.**

8.3 Submission Guidelines

1. **You are responsible for turning in assignments on time. This includes allowing for unforeseen circumstances. If you have an emergency let us know IN ADVANCE of the due time supplying documentation (i.e. note from the dean, doctor's note, etc). Extensions will only be granted to those who contact us in advance of the deadline and no extensions will be made after the due date.**
2. **You are also responsible for ensuring that what you turned in is what you meant to turn in. After submitting you should be sure to download your submission into a brand new folder and test if it works. No excuses if you submit the wrong files, what you turn in is what we grade. In addition, your assignment must be turned in via Canvas/Gradescope. Under no circumstances whatsoever we will accept any email submission of an assignment. Note: if you were granted an extension you will still turn in the assignment over Canvas/Gradescope.**

8.4 Syllabus Excerpt on Academic Misconduct

Academic misconduct is taken very seriously in this class. Quizzes, timed labs and the final examination are individual work.

Homework assignments are collaborative, In addition many if not all homework assignments will be evaluated via demo or code review. During this evaluation, you will be expected to be able to explain every aspect of your submission. Homework assignments will also be examined using computer programs to find evidence of unauthorized collaboration.

What is unauthorized collaboration? Each individual programming assignment should be coded by you. You may work with others, but each student should be turning in their own version of the assignment. Submissions that are essentially identical will receive a zero and will be sent to the Dean of Students' Office of Academic Integrity. Submissions that are copies that have been superficially modified to conceal that they are copies are also considered unauthorized collaboration.

You are expressly forbidden to supply a copy of your homework to another student via electronic means. This includes simply e-mailing it to them so they can look at it. If you supply an electronic copy of your homework to another student and they are charged with copying, you will also be charged. This includes storing your code on any site which would allow other parties to obtain your code such as but not limited to public repositories (Github), pastebin, etc. If you would like to use version control, use github.gatech.edu

8.5 Is collaboration allowed?

Collaboration is allowed on a high level, meaning that you may discuss design points and concepts relevant to the homework with your peers, share algorithms and pseudo-code, as well as help each other debug code. What you shouldn't be doing, however, is pair programming where you collaborate with each other on a single instance of the code. Furthermore, sending an electronic copy of your homework to another student for them to look at and figure out what is wrong with their code is not an acceptable way to help them, because it is frequently the case that the recipient will simply modify the code and submit it as their own.

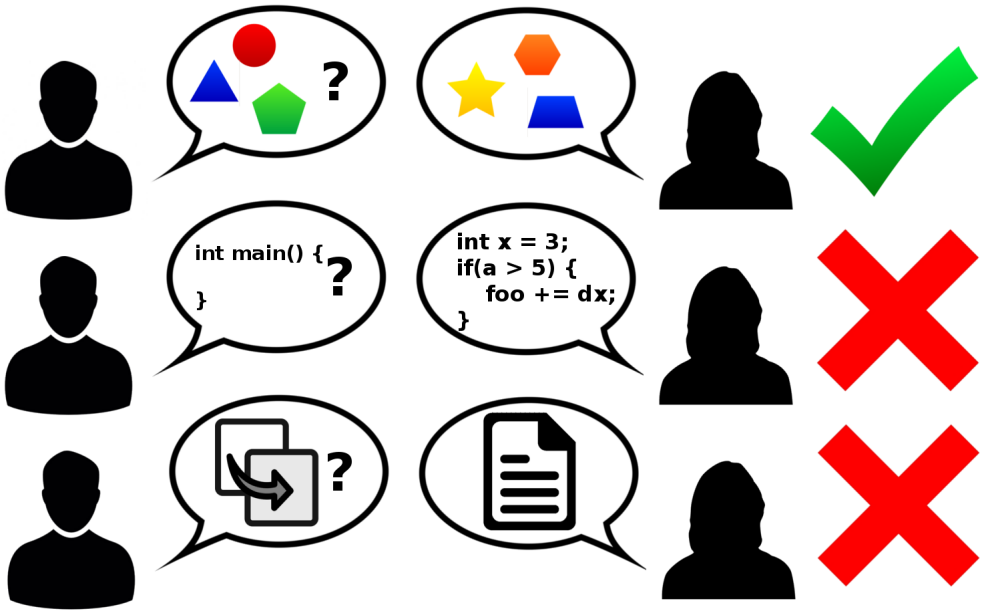


Figure 1: Collaboration rules, explained colorfully